Machine Learning Assignment

1. Perform essential data preprocessing steps on a dataset, including feature scaling, label encoding, data normalization, and train-test splitting.

Python Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder

# 1. Create a sample dataset
np.random.seed(42)
data = {
    'Feature_A': np.random.rand(100) * 100,  # Numerical feature
    'Feature_B': np.random.randint(0, 10, 100), # Numerical feature
    'Category': np.random.choice(['Red', 'Green', 'Blue'], 100), # Categorical feature
    'Target': np.random.randint(0, 2, 100)  # Binary target variable
}
df = pd.DataFrame(data)

print("Original DataFrame head:")
print(df.head())
print("\nOriginal DataFrame info:")
print(df.info())

# Separate features (X) and target (y)
X = df.drop('Target', axis=1)
y = df['Target']

# Identify numerical and categorical features
numerical_cols = ['Feature_A', 'Feature_B']
categorical_cols = ['Category']

# 2. Label Encoding for categorical features (if applicable for target or ordinal categories)
# For demonstration, we'll encode the 'Category' feature
le = LabelEncoder()
X['Category_Encoded'] = le.fit_transform(X['Category'])
# Also add to the original DataFrame for consistent plotting later
df['Category_Encoded'] = le.transform(df['Category'])
print(f"\nLabelEncoder classes for 'Category': {le.classes_}")

# Drop original categorical column if encoded version is used for modeling
X_processed = X.drop('Category', axis=1)

# 3. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_processed, y, test_size=0.2, random_state=42)

print(f"\nTrain set shape: {X_train.shape}")
print(f"Test set shape: {X_test.shape}")

# 4. Feature Scaling (StandardScaler for numerical features)
# Fit scaler only on training data to prevent data leakage
scaler_standard = StandardScaler()
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[numerical_cols] = scaler_standard.fit_transform(X_train[numerical_cols])
X_test_scaled[numerical_cols] = scaler_standard.transform(X_test[numerical_cols])

print("\nFeatures after Standard Scaling (Train head):")
print(X_train_scaled.head())

# 5. Data Normalization (MinMaxScaler for numerical features)
# This is an alternative or additional step depending on the model/data
normalizer = MinMaxScaler()
X_train_normalized = X_train.copy()
X_test_normalized = X_test.copy()

X_train_normalized[numerical_cols] = normalizer.fit_transform(X_train[numerical_cols])
X_test_normalized[numerical_cols] = normalizer.transform(X_test[numerical_cols])

print("\nFeatures after Min-Max Normalization (Train head):")
print(X_train_normalized.head())

print("\nPreprocessing steps completed: Label Encoding, Train-Test Split, Feature Scaling (StandardScaler),
and Data Normalization (MinMaxScaler) have been demonstrated.")
```

Output

```
Original DataFrame head:
   Feature_A  Feature_B Category  Target
0  37.454012          6    Green       0
1  95.071431          0     Blue       1
2  73.199394          3     Blue       1
3  59.865848          3    Green       0
4  15.601864          4    Green       1
```

```
Original DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Feature_A  100 non-null    float64
 1   Feature_B  100 non-null    int64
 2   Category   100 non-null    object
 3   Target     100 non-null    int64
dtypes: float64(1), int64(2), object(1)
memory usage: 3.3+ KB
None

LabelEncoder classes for 'Category': ['Blue' 'Green' 'Red']

Train set shape: (80, 3)
Test set shape: (20, 3)

Features after Standard Scaling (Train head):
    Feature_A  Feature_B  Category_Encoded
55   1.558116  -1.078147                 1
88   1.439100  -0.390882                 0
26  -0.921682   1.327281                 1
42  -1.489217  -1.421780                 2
69   1.781348   0.983648                 2

Features after Min-Max Normalization (Train head):
    Feature_A  Feature_B  Category_Encoded
55   0.933753   0.111111                 1
88   0.898433   0.333333                 0
26   0.197838   0.888889                 1
42   0.029415   0.000000                 2
69   1.000000   0.777778                 2

Preprocessing steps completed: Label Encoding, Train-Test Split, Feature Scaling (StandardScaler), and Data
Normalization (MinMaxScaler) have been demonstrated.
```

2. Build a multiple linear regression model to predict target values using multiple features, and evaluate the regression model's performance.

Python Code

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns

# For simplicity, let's use the Standard Scaled data for linear regression
# You could also choose normalized data or original data based on your specific needs.

# Initialize and train the Linear Regression model
model = LinearRegression()
model.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test_scaled)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"\nMultiple Linear Regression Model Performance:")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"R-squared (R2): {r2:.4f}")

print("\nModel Coefficients:")
for feature, coef in zip(X_train_scaled.columns, model.coef_):
    print(f"{feature}: {coef:.4f}")
print(f"Intercept: {model.intercept_:.4f}")

# Visualize predictions vs actual values
plt.figure(figsize=(10, 6))
sns.regplot(x=y_test, y=y_pred, scatter_kws={'alpha':0.6})
plt.xlabel("Actual Target Values")
plt.ylabel("Predicted Target Values")
plt.title("Actual vs. Predicted Target Values (Linear Regression)")
plt.grid(True)
plt.show()

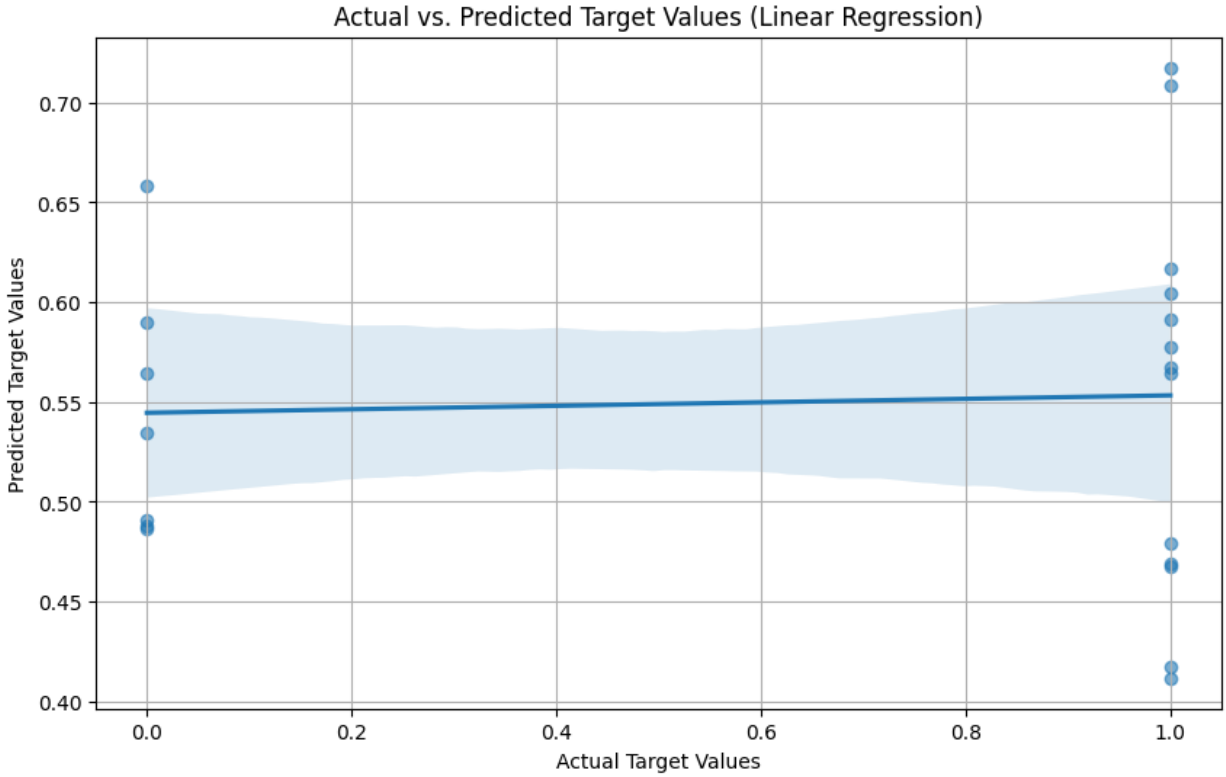
# Visualize residuals
residuals = y_test - y_pred
plt.figure(figsize=(10, 6))
sns.histplot(residuals, kde=True)
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.title("Distribution of Residuals")
plt.grid(True)
```

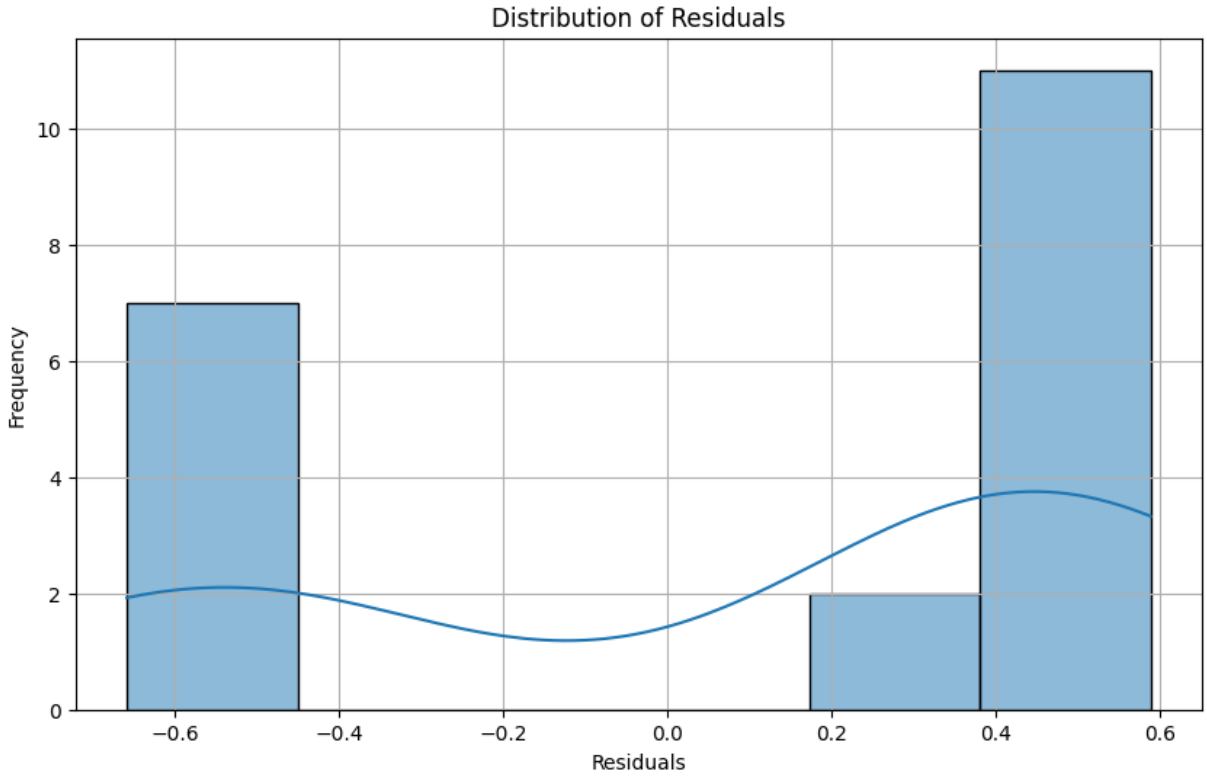
```
plt.show()
```

Output

```
Multiple Linear Regression Model Performance:
Mean Squared Error (MSE): 0.2407
R-squared (R2): -0.0579

Model Coefficients:
Feature_A: -0.0093
Feature_B: 0.0282
Category_Encoded: -0.1017
Intercept: 0.6705
```





3. Apply logistic regression to a binary classification problem, such as student pass/fail prediction, and generate a confusion matrix.

Python Code

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score

# Initialize and train the Logistic Regression model
# Using solver='liblinear' for small datasets or L1/L2 regularization
logistic_model = LogisticRegression(solver='liblinear', random_state=42)
logistic_model.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred_logistic = logistic_model.predict(X_test_scaled)

# Evaluate the Logistic Regression model
print("\n--- Logistic Regression Model Performance ---")

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred_logistic)
print("\nConfusion Matrix:")
print(conf_matrix)

# Visualize Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for Logistic Regression')
plt.show()

# Other classification metrics
accuracy = accuracy_score(y_test, y_pred_logistic)
precision = precision_score(y_test, y_pred_logistic)
recall = recall_score(y_test, y_pred_logistic)
f1 = f1_score(y_test, y_pred_logistic)

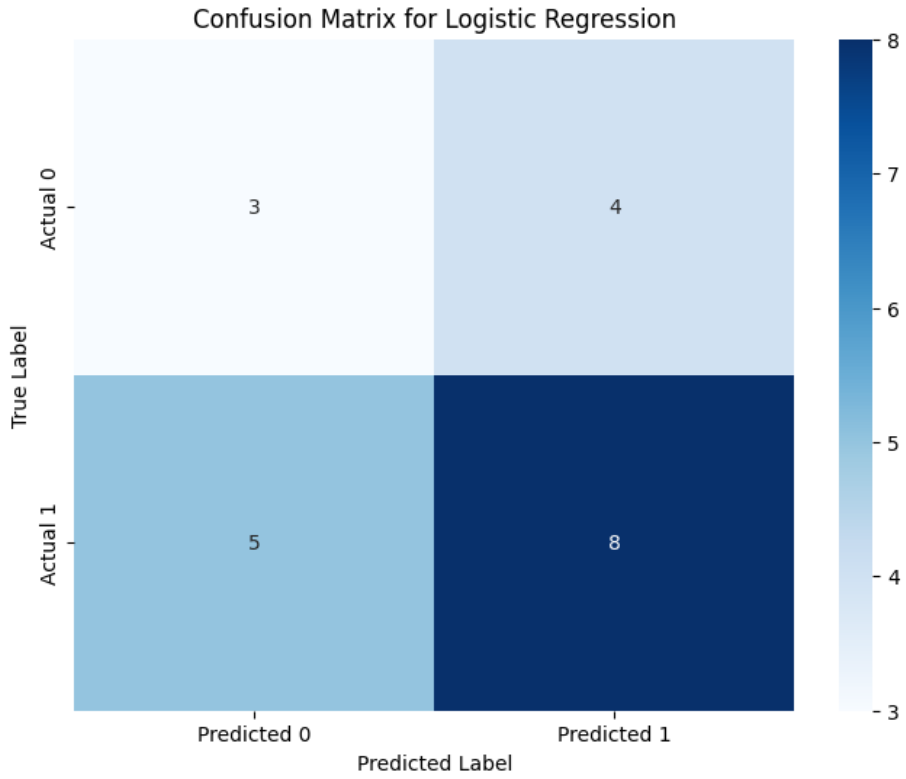
print(f"\nAccuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")

print("\nLogistic Regression model training and evaluation completed.")
```

Output

```
--- Logistic Regression Model Performance ---

Confusion Matrix:
[[3 4]
 [5 8]]
```



Output

```
Accuracy: 0.5500
Precision: 0.6667
Recall: 0.6154
F1-Score: 0.6400

Logistic Regression model training and evaluation completed.
```

4. Perform a classification task using the K-Nearest Neighbors (KNN) algorithm and compare the accuracy for different K values.

Python Code

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import numpy as np

# List to store accuracy for different K values
k_values = range(1, 15) # Test K from 1 to 14
accuracies = []

print("\n--- K-Nearest Neighbors (KNN) Model Performance ---")
for k in k_values:
    # Initialize and train the KNN model
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    # Make predictions on the test set
    y_pred_knn = knn.predict(X_test_scaled)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, y_pred_knn)
    accuracies.append(accuracy)
    print(f"Accuracy for K = {k}: {accuracy:.4f}")

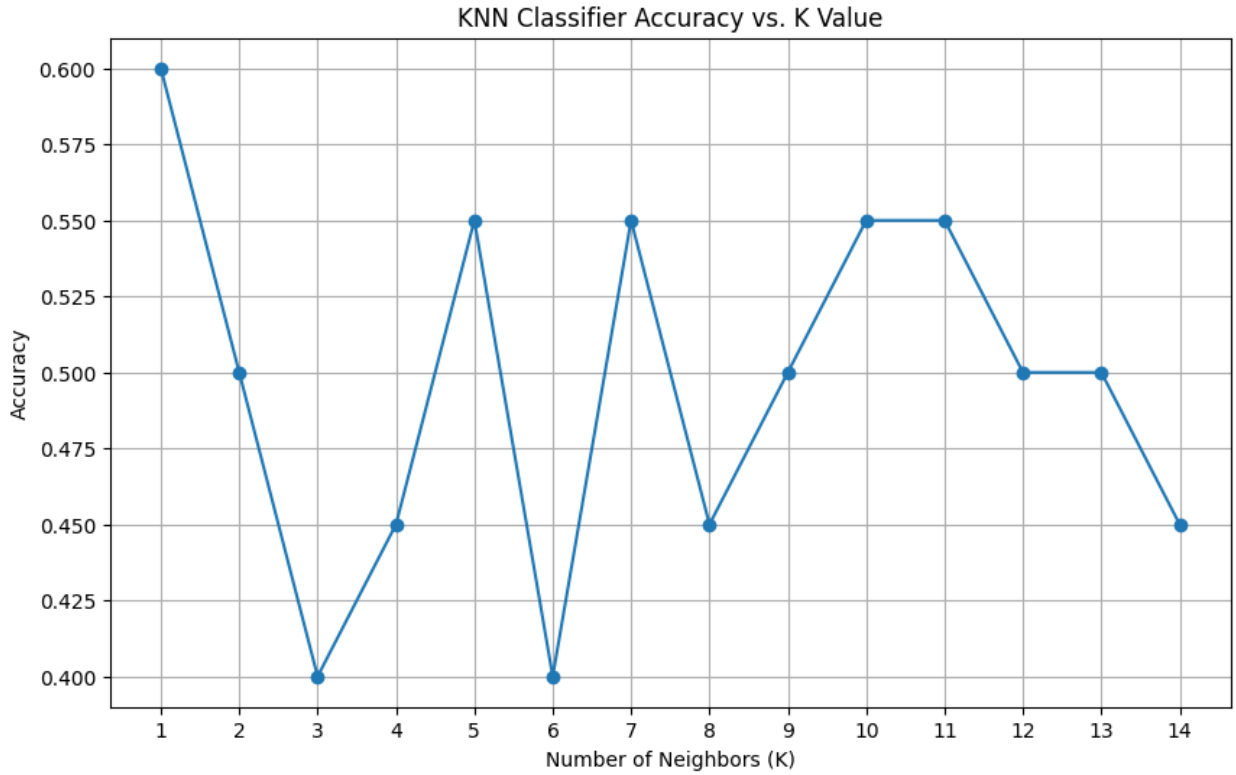
# Plotting the accuracy for different K values
plt.figure(figsize=(10, 6))
plt.plot(k_values, accuracies, marker='o', linestyle='-')
plt.title('KNN Classifier Accuracy vs. K Value')
plt.xlabel('Number of Neighbors (K)')
plt.ylabel('Accuracy')
plt.xticks(list(k_values))
plt.grid(True)
plt.show()

# Find the best K and its accuracy
best_k = k_values[np.argmax(accuracies)]
best_accuracy = np.max(accuracies)
```

```
print(f"\nOptimal K value: {best_k} with an accuracy of {best_accuracy:.4f}")
```

Output

```
--- K-Nearest Neighbors (KNN) Model Performance ---
Accuracy for K = 1: 0.6000
Accuracy for K = 2: 0.5000
Accuracy for K = 3: 0.4000
Accuracy for K = 4: 0.4500
Accuracy for K = 5: 0.5500
Accuracy for K = 6: 0.4000
Accuracy for K = 7: 0.5500
Accuracy for K = 8: 0.4500
Accuracy for K = 9: 0.5000
Accuracy for K = 10: 0.5500
Accuracy for K = 11: 0.5500
Accuracy for K = 12: 0.5000
Accuracy for K = 13: 0.5000
Accuracy for K = 14: 0.4500
```



Output

```
Optimal K value: 1 with an accuracy of 0.6000
```

5. Build a decision tree classifier model for a dataset and visualize the resulting decision tree.

Python Code

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

print("\n--- Decision Tree Classifier Model Performance ---")

# Initialize and train the Decision Tree Classifier
# For simplicity, we'll use default parameters, but these can be tuned.
decision_tree = DecisionTreeClassifier(random_state=42)
decision_tree.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred_dt = decision_tree.predict(X_test_scaled)

# Evaluate the model
accuracy_dt = accuracy_score(y_test, y_pred_dt)
print(f"\nAccuracy of Decision Tree: {accuracy_dt:.4f}")
print("\nClassification Report for Decision Tree:")
print(classification_report(y_test, y_pred_dt))

# Visualize the Decision Tree (can be large for complex trees)
plt.figure(figsize=(20, 15))
plot_tree(decision_tree, filled=True, feature_names=X_train_scaled.columns, class_names=['Fail', 'Pass'],
```

```
rounded=True, fontsize=10)
plt.title('Decision Tree Classifier Visualization')
plt.show()
```

Output

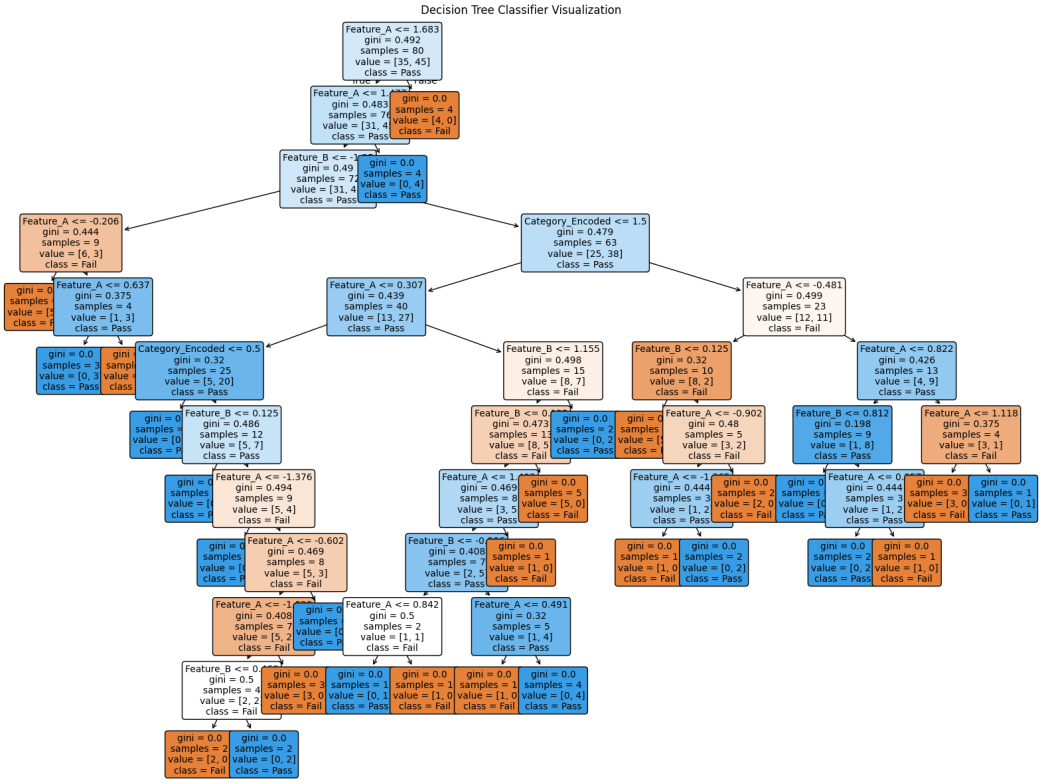
```
--- Decision Tree Classifier Model Performance ---

Accuracy of Decision Tree: 0.5500

Classification Report for Decision Tree:
              precision    recall  f1-score   support

           0       0.38      0.43      0.40         7
           1       0.67      0.62      0.64        13

    accuracy                           0.55        20
   macro avg       0.52      0.52      0.52        20
weighted avg       0.56      0.55      0.56        20
```



6. Apply the K-Means clustering algorithm for customer segmentation and visualize the generated clusters.

Python Code

```
from sklearn.cluster import KMeans
import seaborn as sns

print("\n--- K-Means Clustering for Customer Segmentation ---")

# Determine an appropriate number of clusters. Let's assume 3 for demonstration,
# based on the 3 categories in the original data. In a real scenario, you'd use
# methods like the Elbow method or Silhouette score to find the optimal k.
n_clusters = 3

# Initialize and fit the K-Means model
kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10) # n_init is explicitly set to avoid future
warnings
kmeans.fit(X_processed) # Using X_processed for clustering since it's unsupervised

# Get cluster assignments
clusters = kmeans.predict(X_processed)
df['Cluster'] = clusters # Add cluster assignments to the original DataFrame

print(f"\nNumber of clusters: {n_clusters}")
print("\nCluster distribution:")
print(df['Cluster'].value_counts())

# Visualize the clusters
# Since we have 3 features, we'll visualize in 2D using Feature_A and Feature_B,
# and use color to represent clusters.
plt.figure(figsize=(10, 8))
```

```
sns.scatterplot(x='Feature_A', y='Feature_B', hue='Cluster', data=df, palette='viridis', s=100, alpha=0.8)
plt.title(f'K-Means Clustering with K={n_clusters} (Features A vs B)')
plt.xlabel('Feature_A (Original Scale)')
plt.ylabel('Feature_B (Original Scale)')
plt.grid(True)
plt.show()

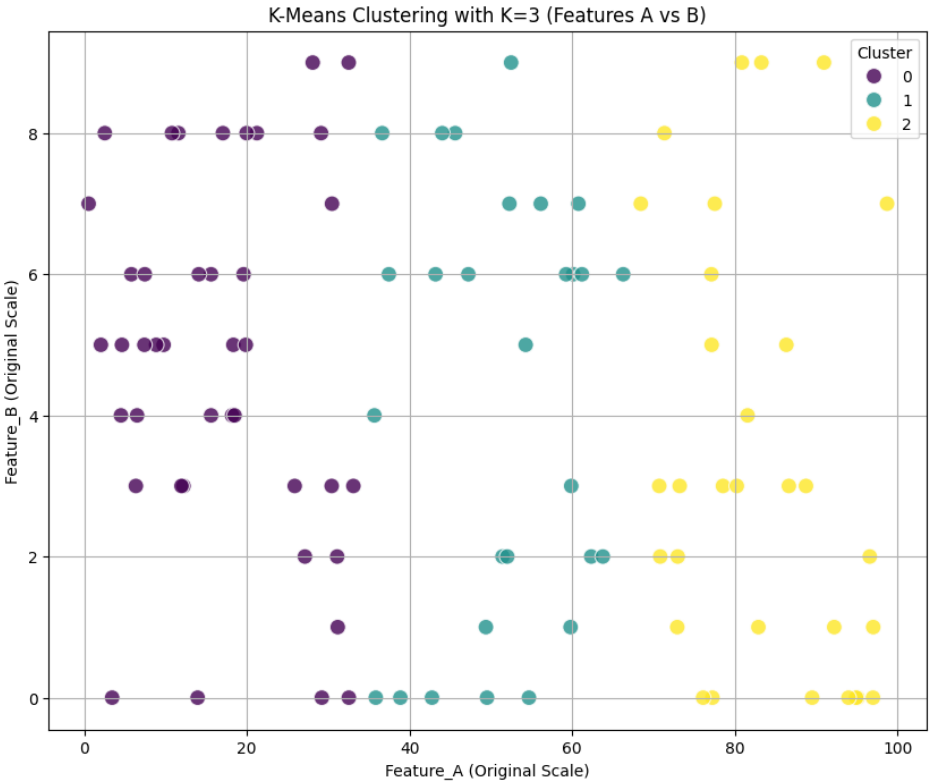
# Another visualization using Category_Encoded if desired, against one of the other features
plt.figure(figsize=(10, 8))
sns.scatterplot(x='Feature_A', y='Category_Encoded', hue='Cluster', data=df, palette='viridis', s=100,
alpha=0.8)
plt.title(f'K-Means Clustering with K={n_clusters} (Features A vs Encoded Category)')
plt.xlabel('Feature_A (Original Scale)')
plt.ylabel('Category_Encoded')
plt.yticks(ticks=np.arange(len(le.classes_)), labels=le.classes_) # Use actual category names if available
plt.grid(True)
plt.show()
```

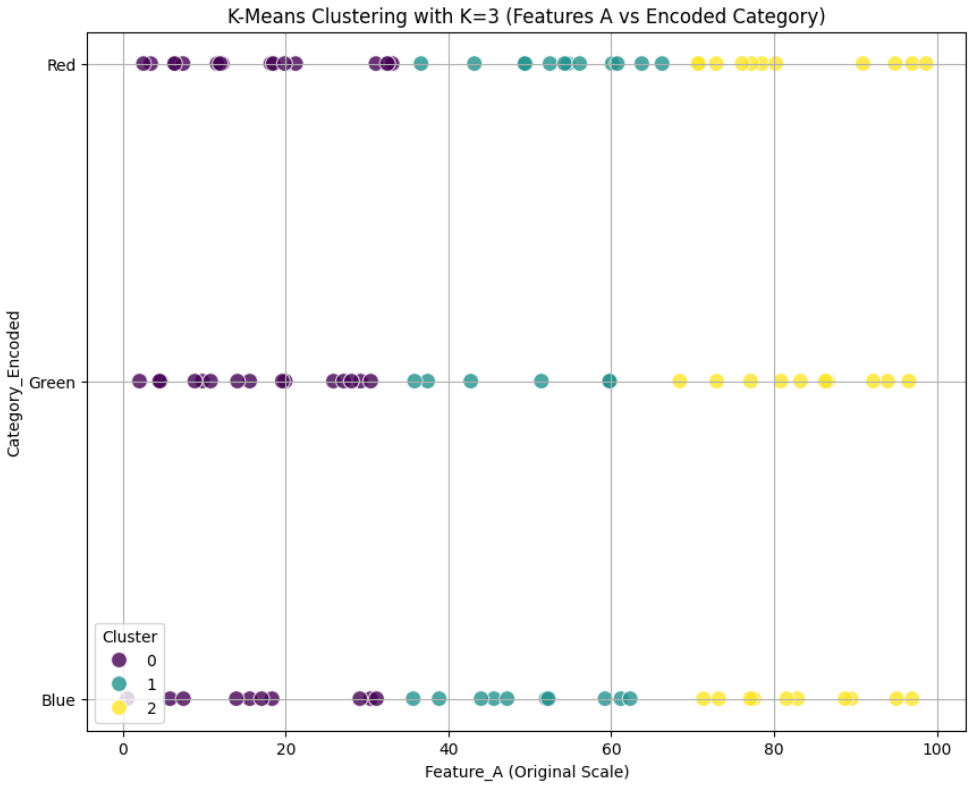
Output

```
--- K-Means Clustering for Customer Segmentation ---

Number of clusters: 3

Cluster distribution:
Cluster
0    41
2    31
1    28
Name: count, dtype: int64
```





7. Write a python program for Student Pass/Fail Classification Using Support Vector Machine (SVM).

Python Code

```
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

print("\n--- Support Vector Machine (SVM) Model Performance ---")

# Initialize and train the Support Vector Machine (SVM) Classifier
# Using a linear kernel for simplicity, but RBF or others can be explored.
svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test_scaled)

# Evaluate the model
accuracy_svm = accuracy_score(y_test, y_pred_svm)
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)

print(f"\nAccuracy of SVM: {accuracy_svm:.4f}")
print("\nConfusion Matrix for SVM:")
print(conf_matrix_svm)
print("\nClassification Report for SVM:")
print(classification_report(y_test, y_pred_svm))

# Visualize Confusion Matrix for SVM
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Predicted 0', 'Predicted 1'],
            yticklabels=['Actual 0', 'Actual 1'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for SVM Classification')
plt.show()
```

Output

```
--- Support Vector Machine (SVM) Model Performance ---

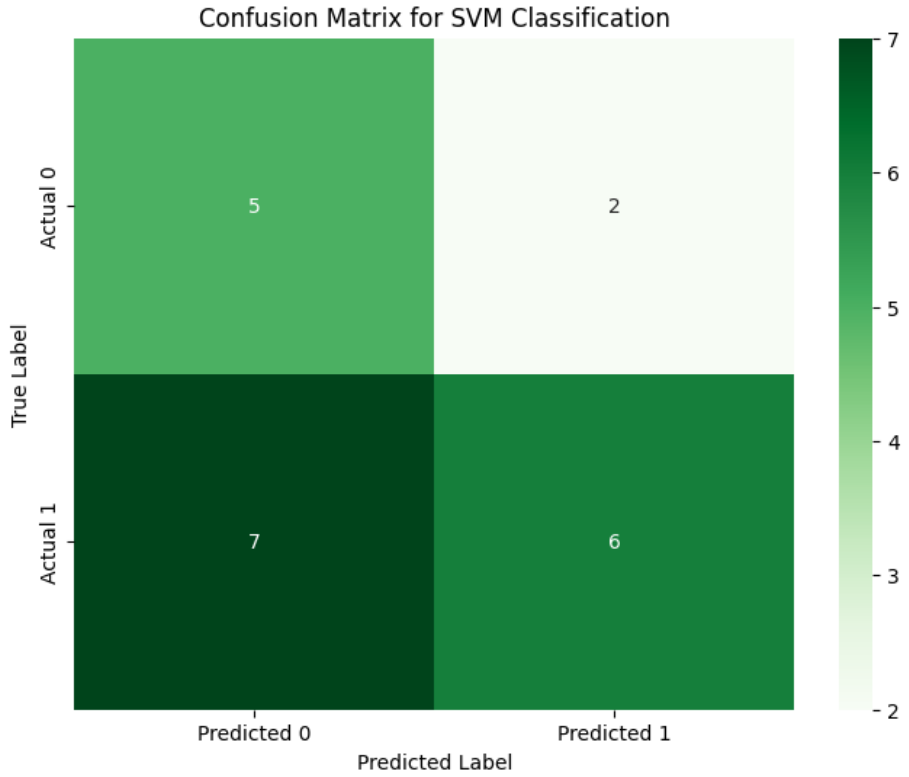
Accuracy of SVM: 0.5500

Confusion Matrix for SVM:
[[5 2]
 [7 6]]

Classification Report for SVM:
              precision    recall  f1-score   support
```

```
              0       0.42      0.71      0.53         7
              1       0.75      0.46      0.57        13

    accuracy                            0.55        20
   macro avg       0.58      0.59      0.55        20
weighted avg       0.63      0.55      0.56        20
```



8. Complete a mini-project using a real-world dataset on one of the suggested topics: Movie recommendation system.

Python Code

```
import os, re, urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.sparse as sp
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.neighbors import NearestNeighbors
from sklearn.decomposition import TruncatedSVD
import warnings; warnings.filterwarnings("ignore")
%matplotlib inline
np.random.seed(42)
```

Python Code

```
BASE = "https://raw.githubusercontent.com/sidooms/MovieTweetings/master/latest/"

def load(fname, cols):
    if not os.path.exists(fname):                      # download once, then reuse local copy
        urllib.request.urlretrieve(BASE + fname, fname)
    return pd.read_csv(fname, sep="::", engine="python", names=cols,
                       dtype={"movie_id": str}, encoding_errors="ignore")

movies  = load("movies.dat",  ["movie_id", "title", "genres"])
ratings = load("ratings.dat", ["user_id", "movie_id", "rating", "ts"])
print("movies :", movies.shape)
print("ratings:", ratings.shape)
display(movies.head())
display(ratings.head())
```

Output

```
movies : (38018, 3)
ratings: (921398, 4)
```

Output

```
  movie_id                                        title  \
0  0000008      Edison Kinetoscopic Record of a Sneeze (1894)
1  0000010                 La sortie des usines Lumière (1895)
```

```
2  0000012                     The Arrival of a Train (1896)
3       25  The Oxford and Cambridge University Boat Race ...
4  0000091                        Le manoir du diable (1896)

             genres
0  Documentary|Short
1  Documentary|Short
2  Documentary|Short
3               NaN
4      Short|Horror
```

Output

```
   user_id movie_id  rating          ts
0        1  0114508       8  1381006850
1        2  0499549       9  1376753198
2        2  1305591       8  1376742507
3        2  1428538       1  1371307089
4        3  0075314       1  1595468524
```

Python Code

```
# Extract release year, clean genres, drop duplicates / missing
movies["year"]   = movies.title.str.extract(r"\((\d{4})\)\s*$").astype(float)
movies["genres"] = movies.genres.fillna("")
movies = movies.drop_duplicates("movie_id")
ratings = ratings.drop_duplicates(["user_id", "movie_id"]).dropna()
print("Missing values in ratings:", int(ratings.isna().sum().sum()))
print("Unique users :", ratings.user_id.nunique(), "| Unique rated movies:", ratings.movie_id.nunique())
sparsity = 1 - len(ratings) / (ratings.user_id.nunique() * ratings.movie_id.nunique())
print(f"Rating-matrix sparsity: {sparsity:.4%}  (that is why recommenders are hard!)")
```

Output

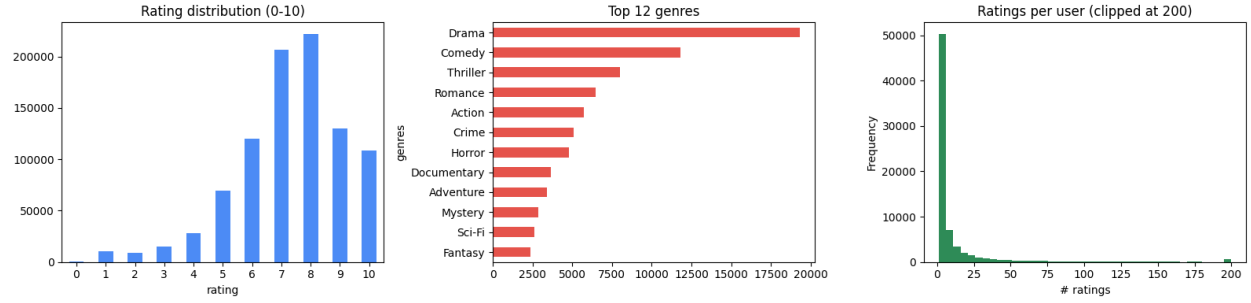
```
Missing values in ratings: 0
Unique users : 71707 | Unique rated movies: 38018
Rating-matrix sparsity: 99.9662%  (that is why recommenders are hard!)
```

Python Code

```
fig, ax = plt.subplots(1, 3, figsize=(16, 4))
ratings.rating.value_counts().sort_index().plot.bar(ax=ax[0], color="#4c8bf5", rot=0)
ax[0].set_title("Rating distribution (0-10)"); ax[0].set_xlabel("rating")

genre_counts = movies.genres.str.split("|").explode().value_counts().head(12)
genre_counts.sort_values().plot.barh(ax=ax[1], color="#e5534b"); ax[1].set_title("Top 12 genres")

ratings.groupby("user_id").size().clip(upper=200).plot.hist(bins=40, ax=ax[2], color="#2e8b57")
ax[2].set_title("Ratings per user (clipped at 200)"); ax[2].set_xlabel("# ratings")
plt.tight_layout(); plt.show()
print("Average rating:", round(ratings.rating.mean(), 2))
```



Output

```
Average rating: 7.31
```

Python Code

```
df = ratings.copy()
for _ in range(3):
    mc = df.movie_id.value_counts(); df = df[df.movie_id.isin(mc[mc >= 100].index)]
    uc = df.user_id.value_counts();  df = df[df.user_id.isin(uc[uc >= 20].index)]

# integer indices for matrix building
df["u"] = df.user_id.astype("category").cat.codes
df["i"] = df.movie_id.astype("category").cat.codes
movie_ids = df.movie_id.astype("category").cat.categories
n_users, n_items = df.u.max() + 1, df.i.max() + 1

meta = (movies.set_index("movie_id").loc[movie_ids].reset_index())

# Fix: Ensure 'movie_id' column exists, if 'index' was created instead
if 'index' in meta.columns and 'movie_id' not in meta.columns:
    meta = meta.rename(columns={'index': 'movie_id'})

# Add the 'i' column to meta by mapping from df
```

```
movie_id_to_i_map = df.drop_duplicates('movie_id').set_index('movie_id')['i']
meta['i'] = meta['movie_id'].map(movie_id_to_i_map)

meta["n_ratings"] = df.groupby("i").size().reindex(range(n_items)).values
meta["avg_rating"] = df.groupby("i").rating.mean().reindex(range(n_items)).values
print(f"Kept {len(df):,} ratings | {n_users:,} users | {n_items:,} movies")

top = meta.sort_values("n_ratings", ascending=False).head(10)[["title", "genres", "n_ratings", "avg_rating"]]
display(top.round(2))
```

Output

```
Kept 412,517 ratings | 6,922 users | 1,357 movies
```

Output

```
                           title                                     genres  \
555                  Gravity (2013)                      Drama|Sci-Fi|Thriller
373  The Wolf of Wall Street (2013)                      Biography|Crime|Drama
329             Interstellar (2014)                    Adventure|Drama|Sci-Fi
878                Gone Girl (2014)                     Drama|Mystery|Thriller
651             The Revenant (2015)  Action|Adventure|Biography|Drama|Western
546                 Deadpool (2016)            Action|Adventure|Comedy|Sci-Fi
586         Captain Phillips (2013)                 Biography|Drama|Thriller
310             Man of Steel (2013)                  Action|Adventure|Sci-Fi
527                Prisoners (2013)              Crime|Drama|Mystery|Thriller
526       Mad Max: Fury Road (2015)          Action|Adventure|Sci-Fi|Thriller

     n_ratings  avg_rating
555       1742        8.16
373       1710        8.26
329       1552        8.78
878       1511        8.18
651       1405        8.12
546       1370        7.99
586       1360        8.14
310       1344        7.43
527       1341        8.09
526       1338        7.99
```

Python Code

```
def find_movie(text, n=8):
    """Search helper: returns index (row in meta) of movies whose title contains `text`."""
    hits = meta[meta.title.str.contains(text, case=False, regex=False)].sort_values("n_ratings",
ascending=False)
    return hits.head(n)

display(find_movie("Inception")[["title", "year", "n_ratings"]])
display(find_movie("Toy Story")[["title", "year", "n_ratings"]])
```

Output

```
               title    year  n_ratings
517  Inception (2010)  2010.0        588
```

Output

```
                 title    year  n_ratings
780  Toy Story 4 (2019)  2019.0        357
263  Toy Story 3 (2010)  2010.0        160
114    Toy Story (1995)  1995.0        156
137  Toy Story 2 (1999)  1999.0        112
```

Python Code

```
# ---- train / test split ----
rng = np.random.default_rng(42)
mask = rng.random(len(df)) < 0.8
train, test = df[mask].copy(), df[~mask].copy()
print(f"train={len(train):,}  test={len(test):,}")

g_mean = train.rating.mean()
u_mean = train.groupby("u").rating.mean().reindex(range(n_users)).fillna(g_mean).values
i_mean = train.groupby("i").rating.mean().reindex(range(n_items)).fillna(g_mean).values

i_dev = i_mean - g_mean                                  # how much better/worse a movie is than average
base_train = u_mean[train.u.values] + i_dev[train.i.values]   # baseline = user bias + movie bias
train["c"] = train.rating - base_train
R = sp.csr_matrix((train.c.values, (train.u.values, train.i.values)), shape=(n_users, n_items))

def rmse_mae(pred):
    err = test.rating.values - pred
    return np.sqrt((err**2).mean()), np.abs(err).mean()

results = {}
results["Global mean"] = rmse_mae(np.full(len(test), g_mean))
results["User mean"]   = rmse_mae(u_mean[test.u.values])
results["Item mean"]   = rmse_mae(i_mean[test.i.values])
base_test = u_mean[test.u.values] + i_dev[test.i.values]
results["Baseline (user+item bias)"] = rmse_mae(np.clip(base_test, 0, 10))

svd_models = {}
```

```
for k in [5, 10, 20, 40, 80]:
    svd = TruncatedSVD(n_components=k, random_state=42).fit(R)
    U = svd.transform(R)                    # user factors (n_users x k)
    V = svd.components_                      # movie factors (k x n_items)
    pred = base_test + np.einsum("ij,ji->i", U[test.u.values], V[:, test.i.values])
    results[f"SVD (k={k})"] = rmse_mae(np.clip(pred, 0, 10))
    svd_models[k] = (U, V)

res = pd.DataFrame(results, index=["RMSE", "MAE"]).T.round(4)
display(res)
```

Output

```
train=330,194  test=82,323
```

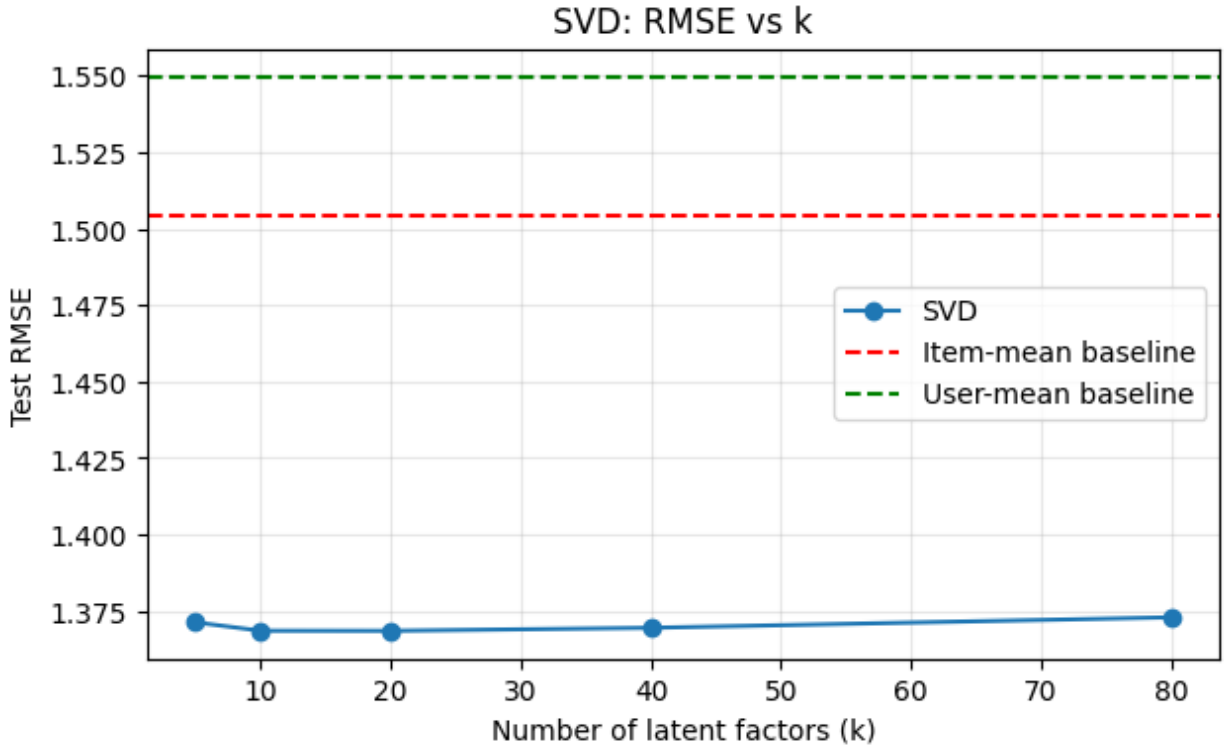
Output

	RMSE	MAE
Global mean	1.7078	1.3308
User mean	1.5498	1.1702
Item mean	1.5044	1.1227
Baseline (user+item bias)	1.3878	1.0201
SVD (k=5)	1.3713	1.0057
SVD (k=10)	1.3686	1.0030
SVD (k=20)	1.3685	1.0027
SVD (k=40)	1.3695	1.0035
SVD (k=80)	1.3729	1.0061

Python Code

```
ks = [5, 10, 20, 40, 80]
plt.figure(figsize=(7, 4))
plt.plot(ks, [results[f"SVD (k={k})"][0] for k in ks], marker="o", label="SVD")
plt.axhline(results["Item mean"][0], color="r", ls="--", label="Item-mean baseline")
plt.axhline(results["User mean"][0], color="g", ls="--", label="User-mean baseline")
plt.xlabel("Number of latent factors (k)"); plt.ylabel("Test RMSE"); plt.title("SVD: RMSE vs k")
plt.legend(); plt.grid(alpha=.3); plt.show()

best_k = min(ks, key=lambda k: results[f"SVD (k={k})"][0])
print("Best k by RMSE:", best_k)
```



Output

```
Best k by RMSE: 20
```

Python Code

```
# ---- Precision@10: ranking quality ----
test_rel = test[test.rating >= 8].groupby("u").i.apply(set)
sample_users = rng.choice(test_rel.index.values, size=min(2000, len(test_rel)), replace=False)

seen = train.groupby("u").i.apply(np.array)
pop_order = train.groupby("i").size().reindex(range(n_items)).fillna(0).values

def precision_at_k(score_fn, k=10):
```

```
    hits = 0
    for u in sample_users:
        s = score_fn(u).copy()
        s[seen.get(u, [])] = -np.inf                  # never recommend already-rated movies
        top = np.argpartition(-s, k)[:k]
        hits += len(set(top) & test_rel[u])
    return hits / (k * len(sample_users))

# (a) popularity baseline
p_pop = precision_at_k(lambda u: pop_order.astype(float))

# (b) rating-prediction SVD used for ranking (shows why a different model is needed for top-N)
U, V = svd_models[best_k]
p_rating_svd = precision_at_k(lambda u: u_mean[u] + i_dev + U[u] @ V)

# (c) implicit-feedback SVD on the binary "who rated what" matrix
B = sp.csr_matrix((np.ones(len(train)), (train.u.values, train.i.values)), shape=(n_users, n_items))
rank_scores = {}
for k in [10, 20, 50, 100]:
    svd_i = TruncatedSVD(n_components=k, random_state=42).fit(B)
    Ui, Vi = svd_i.transform(B), svd_i.components_
    rank_scores[k] = precision_at_k(lambda u: Ui[u] @ Vi)
    print(f"Implicit SVD k={k:<3} Precision@10 = {rank_scores[k]:.4f}")
best_k_rank = max(rank_scores, key=rank_scores.get)

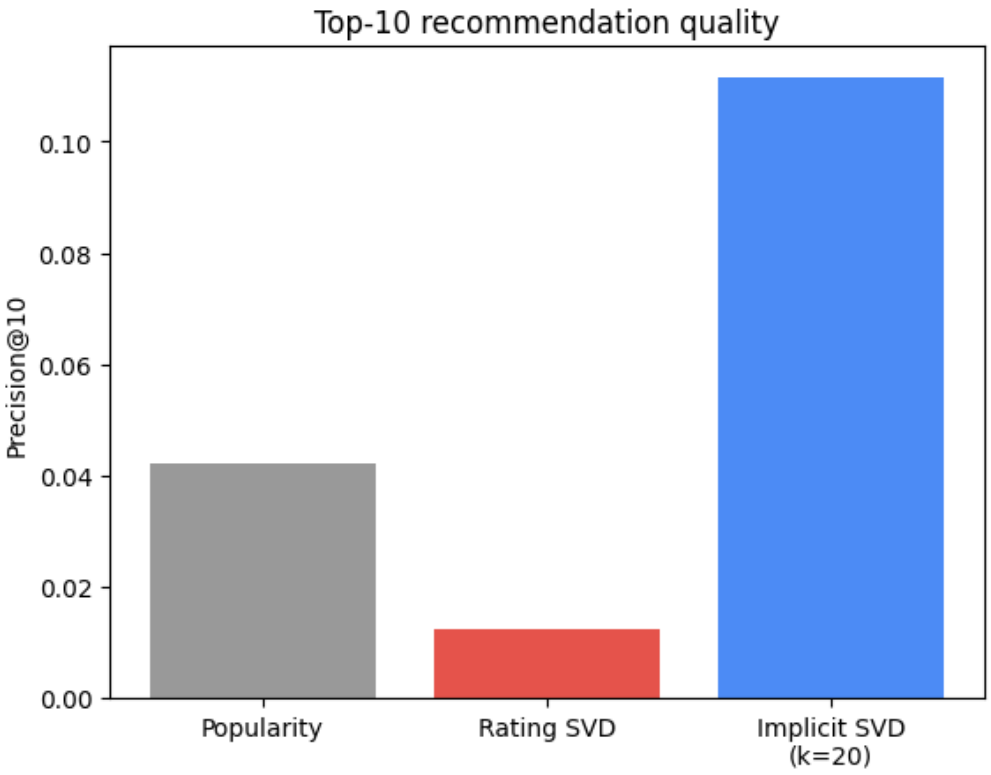
print(f"\nPopularity baseline              : {p_pop:.4f}")
print(f"Rating-prediction SVD (k={best_k})     : {p_rating_svd:.4f}")
print(f"Implicit SVD for ranking (k={best_k_rank}) : {rank_scores[best_k_rank]:.4f}")

plt.bar(["Popularity", "Rating SVD", f"Implicit SVD\n(k={best_k_rank})"],
        [p_pop, p_rating_svd, rank_scores[best_k_rank]], color=["#999", "#e5534b", "#4c8bf5"])
plt.ylabel("Precision@10"); plt.title("Top-10 recommendation quality"); plt.show()
```

Output

```
Implicit SVD k=10  Precision@10 = 0.1018
Implicit SVD k=20  Precision@10 = 0.1115
Implicit SVD k=50  Precision@10 = 0.1005
Implicit SVD k=100 Precision@10 = 0.0849

Popularity baseline              : 0.0422
Rating-prediction SVD (k=20)     : 0.0124
Implicit SVD for ranking (k=20) : 0.1115
```



Python Code

```
g_f  = df.rating.mean()
um_f = df.groupby("u").rating.mean().reindex(range(n_users)).fillna(g_f).values
im_dev_f = meta.avg_rating.values - g_f                    # movie bias on all data
full_c = df.rating.values - (um_f[df.u.values] + im_dev_f[df.i.values])
R_full = sp.csr_matrix((full_c, (df.u.values, df.i.values)), shape=(n_users, n_items))
svd_f = TruncatedSVD(n_components=best_k, random_state=42).fit(R_full)         # rating prediction
U_f, V_f = svd_f.transform(R_full), svd_f.components_

B_full = sp.csr_matrix((np.ones(len(df)), (df.u.values, df.i.values)), shape=(n_users, n_items))
```

```
svd_r = TruncatedSVD(n_components=best_k_rank, random_state=42).fit(B_full)    # ranking
U_r, V_r = svd_r.transform(B_full), svd_r.components_
seen_full = df.groupby("u").i.apply(np.array)

def predict_user(u):
    return np.clip(um_f[u] + im_dev_f + U_f[u] @ V_f, 0, 10)

def affinity(u):
    return U_r[u] @ V_r                                  # how strongly the user's taste matches each movie

def recommend_for_user(u, n=10):
    scores = affinity(u)
    scores[seen_full[u]] = -np.inf                       # hide already-rated movies
    top = np.argsort(-scores)[:n]
    out = meta.iloc[top][["title", "genres", "avg_rating"]].copy()
    out["predicted_rating"] = predict_user(u)[top]
    return out.round(2)

def show_user_history(u, n=8):
    h = df[df.u == u].sort_values("rating", ascending=False).head(n)
    return pd.DataFrame({"title": meta.title.iloc[h.i.values].values, "your_rating": h.rating.values})

# Select the user with the most ratings to ensure a rich history and recommendations
demo_user = df.groupby("u").size().idxmax()
print(f"User #{demo_user} - their top-rated movies:"); display(show_user_history(demo_user))
print("Recommended for this user:"); display(recommend_for_user(demo_user))
```

Output

```
User #1637 - their top-rated movies:
```

Output

```
                      title  your_rating
0         The Martian (2015)           10
1    Captain Phillips (2013)           10
2       Winter's Bone (2010)           10
3  Mad Max: Fury Road (2015)           10
4    Zero Dark Thirty (2012)           10
5             Titanic (1997)           10
6    Schindler's List (1993)           10
7           Ah-ga-ssi (2016)           10
```

Output

```
Recommended for this user:
```

Output

```
                                 title                                genres  \
835              Saving Mr. Banks (2013)         Biography|Comedy|Drama|Music
851                    About Time (2013)  Comedy|Drama|Fantasy|Romance|Sci-Fi
230   A Walk Among the Tombstones (2014)          Action|Crime|Drama|Thriller
585                       Elysium (2013)                 Action|Drama|Sci-Fi
1024                         Lucy (2014)              Action|Sci-Fi|Thriller
495          August: Osage County (2013)                               Drama
903                 Blue Jasmine (2013)                               Drama
950                    Philomena (2013)               Biography|Comedy|Drama
631              Edge of Tomorrow (2014)                        Action|Sci-Fi
931                  Delivery Man (2013)                         Comedy|Drama

      avg_rating  predicted_rating
835         7.78              7.53
851         7.74              7.52
230         6.45              6.36
585         6.82              5.59
1024        6.38              6.71
495         7.35              7.20
903         7.31              7.56
950         7.86              7.82
631         7.82              8.08
931         6.42              5.91
```

Now, let's add a function to recommend movies similar to a given movie title. This will use the movie's latent factors derived from the SVD model to find other movies with similar characteristics.

Python Code

```
from sklearn.metrics.pairwise import cosine_similarity

def recommend_similar_movies(movie_title, n=10):
    """Recommends movies similar to the given movie title."""

    # Find the movie in our metadata
    found_movies = find_movie(movie_title, n=1)
    if found_movies.empty:
        print(f"Movie '{movie_title}' not found in the database.")
        return
```

```
    # Get the internal index of the found movie
    movie_idx = found_movies.index[0] # Assumes `find_movie` returns a DataFrame indexed by meta index
    i_value = meta.loc[movie_idx]['i'] # Get the 'i' (internal movie ID) for the found movie

    # Get the latent factor vector for the target movie from V_r
    target_movie_vector = V_r[:, i_value].reshape(1, -1)

    # Calculate cosine similarity with all other movie vectors
    # V_r has shape (k, n_items), so we need to transpose it to (n_items, k) for similarity calculation
    similarities = cosine_similarity(target_movie_vector, V_r.T)[0]

    # Get top N similar movies (excluding itself)
    # Create a Series for easy sorting and excluding
    similar_movies_scores = pd.Series(similarities, index=meta.index)

    # Exclude the movie itself
    similar_movies_scores = similar_movies_scores.drop(movie_idx)

    # Sort and get top N
    top_similar_indices = similar_movies_scores.nlargest(n).index

    # Retrieve movie details
    recommendations = meta.loc[top_similar_indices].copy()
    recommendations['similarity_score'] = similar_movies_scores.loc[top_similar_indices].round(4)

    print(f"\nMovies similar to '{movie_title}':")
    display(recommendations[['title', 'genres', 'avg_rating', 'similarity_score']])

# Example usage:
movie_name_input = "The Dark Knight"
recommend_similar_movies(movie_name_input)
```

Output

```
Movies similar to 'The Dark Knight':
```

Output

```
                                          title  \
505                      The Dark Knight Rises (2012)
235                             Batman Begins (2005)
136                                   Titanic (1997)
208  Pirates of the Caribbean: The Curse of the Bla...
144  The Lord of the Rings: The Fellowship of the R...
372                             Sherlock Holmes (2009)
159  The Lord of the Rings: The Return of the King ...
298                                I Am Legend (2007)
275    Pirates of the Caribbean: At World's End (2007)
160       The Lord of the Rings: The Two Towers (2002)

                                    genres  avg_rating  similarity_score
505                       Action|Thriller    8.710811            0.9824
235                      Action|Adventure    8.433121            0.9677
136                         Drama|Romance    8.454545            0.9194
208                Action|Adventure|Fantasy    8.228571            0.9071
144                 Adventure|Drama|Fantasy    8.658703            0.9042
372  Action|Adventure|Crime|Mystery|Thriller    7.821192            0.8984
159                 Adventure|Drama|Fantasy    9.272059            0.8962
298                    Drama|Sci-Fi|Thriller    7.544776            0.8911
275                Action|Adventure|Fantasy    7.509091            0.8896
160                 Adventure|Drama|Fantasy    8.833333            0.8849
```